MOBIUS BLOOM VENTURES PVT. LTD. (MobiusEMS / Whalexy)

Multi-Tenant Attendance & Deterministic Payroll Engine

Comprehensive Technical Architecture, Security & Implementation Guide

Candidate: Aditya Bhalerao

Role: Full-Stack Developer (Round 2 Take-Home Assessment)

Date: October 2026

KEY DELIVERABLES INCLUDED IN THIS GUIDE

- End-to-End Multi-Tenant Isolation & Zero-Leaked 404 HTTP Semantics
- Pure Deterministic Payroll Engine & 20 Passing Automated Tests
- Mobile-First React (Vite) UI with Dark/Light Mode & 1-Click Demo Logins
- Bonus Grounded Natural Language Query Assistant (No Hallucinations)
- Detailed Code Tour, Formula Walkthrough & 6 Interview Model Answers

Table of Contents

This reference guide documents the architecture, implementation choices, and operational instructions for the project.

- 1. Project Overview & Requirements Satisfied
- 2. System Architecture & Directory Structure
- 3. Getting Started: Installation, Seeding & Execution
- 4. Demo Credentials & Step-by-Step Reviewer Tour
- 5. Pure Payroll Calculation Engine & Worked Example
- 6. Multi-Tenancy Enforcement & RBAC Security Guarantees
- 7. File-by-File Code Tour & End-to-End Request Flow
- 8. Bonus Natural Language Query Assistant (POST /api/ask)
- 9. Assumptions, Known Limitations & Architectural Trade-offs
- 10. Likely Technical Interview Questions & Model Answers
- 11. Production Cloud Deployment Blueprint (Render + Vercel)

1. Project Overview & Requirements Satisfied

This system was developed as a Round 2 take-home engineering assessment for Mobius Bloom Ventures Pvt. Ltd. (MobiusEMS / Whalexy). It solves the problem of automated attendance aggregation and monthly salary disbursement across multiple corporate tenants.

Core Requirements Fully Met:

- Two distinct companies (Acme Textiles & Bright Foods), each configured with distinct working days (26 and 24 days).
- Deterministic payroll endpoint calculating Loss of Pay (LOP) days, earned gross salary, 10% statutory tax/PF deductions, and final net payouts.
- Role-Based Access Control (RBAC): Admin can manage and run payroll for anyone in their organization; Employee can only view their own records.
- Cryptographic Multi-Tenancy: Company A data is strictly invisible to Company B. Cross-tenant access yields 404 Not Found to prevent resource enumeration.
- Robust backend testing suite with 20 passing unit and integration test cases covering edge cases, rounding, and tenant boundaries.
- Bonus Grounded Natural-Language Query Assistant (POST /api/ask) answering queries like "What is Ramesh's net salary for October?" exclusively via internal tool execution.
- High-polish mobile-first dashboard UI with dark/light theme switching, interactive charts (Recharts), and quick 1-click demo logins.

2. Architecture & Monorepo Directory Structure

The project follows a clean monorepo architecture separating the Node/Express backend from the React Vite frontend.

```
Employee_payroll_system/
% % %  AGENTS.md                 # Complete assignment specification & rules
% % %  README.md                 # High-level repo documentation & guide
% % %  DESIGN.md                 # 2-page system design & security note
% % %  API.md                    # REST API specifications and contracts
% % %  PROGRESS.md               # Development memory, changelog & checklist
% % %  PROJECT_GUIDE.pdf         # Comprehensive project walkthrough document
% % %  .gitignore                # Excludes .env, node_modules, build outputs
%
% % %  server/                   # Backend Express REST API
%    % % %  .env.example          # Template for backend secrets
%    % % %  package.json          # Dependencies (express, mongoose, bcrypt, jwt)
%    % % %  src/
%    %    % % %  config/db.js      # MongoDB Atlas connection lifecycle
%    %    % % %  models/           # Mongoose schemas: Company, User, Attendance
%    %    % % %  controllers/      # auth, company, employee, payroll, ask
%    %    % % %  middleware/       # authenticate, authorizeRoles, errorHandler
%    %    % % %  services/         # payrollQueryService (grounded tool engine)
%    %    % % %  utils/            # Pure payrollCalculator.js engine
%    %    % % %  scripts/seed.js   # Idempotent database seeder
%    % % %  tests/                # Jest test suites (unit + integration)
%
% % %  client/                   # Frontend React (Vite) Single-Page App
    % % %  .env.example          # Template for client API URL
    % % %  package.json          # React 18, Tailwind CSS, Lucide, Recharts
    % % %  vite.config.js        # Vite bundler configuration & /api proxy
    % % %  src/
        % % %  api/client.js     # Axios instance with auto-JWT & 401 redirect
        % % %  context/          # AuthContext, ThemeContext, ToastContext
        % % %  components/       # Reusable Button, Card, Modal, Input, Badge
        % % %  pages/            # Login, Dashboard, Employees, Payroll, AskAI
```

3. Getting Started: Installation, Seeding & Execution

Follow these commands to install, seed, test, and run the entire stack locally:

```
# 1. Install Backend Dependencies
cd server
npm install

# 2. Configure Environment Variables
# Copy server/.env.example to server/.env and verify MONGODB_URI & JWT_SECRET

# 3. Seed MongoDB Atlas Database
npm run seed

# 4. Run Automated Backend Test Suite (20 tests)
npm test

# 5. Start Backend Server (runs on http://localhost:5000)
npm run dev   # or: node src/server.js
```

```
# 6. Install & Launch Frontend (in a new terminal)
cd ../client
npm install
npm run dev   # runs on http://localhost:5173
```

4. Demo Credentials & Step-by-Step Reviewer Tour

The seeder sets up two completely segregated organizations to verify isolation and role permissions. The login page provides 1-Click Quick Demo Login buttons for immediate evaluation.

Seeded Company Accounts:

- Company A (Acme Textiles - 26 Working Days):
- • Admin: admin@acme.com | Password: Password123!
- • Employee 1: ramesh@acme.com | Password: Password123! (Ramesh Kumar, CTC ¹60,000)
- • Employee 2: priya@acme.com | Password: Password123! (Priya Sharma, CTC ¹55,000)
- Company B (Bright Foods - 24 Working Days):
- • Admin: admin@bright.com | Password: Password123!
- • Employee 1: ramesh@bright.com | Password: Password123! (Ramesh Gupta, CTC ¹52,000)

Step-by-Step Verification Script:

- 1. Log in as Acme Admin (click Quick Demo button): Note dashboard analytics showing 26 working days for October, total net payroll, and salary distribution.
- 2. Run Payroll: Go to "Run Payroll" (/payroll). Select Ramesh Kumar. Notice default attendance (22 Present, 2 Absent, 2 Half-day = 26 days). Click "Compute & Save Payroll". Observe exact ¹47,769.24 net salary breakdown.
- 3. Verify Isolation: Log out and log in as Bright Admin (Company B). Notice only Bright Foods employees appear. Ramesh Gupta (¹52,000) is shown, with zero visibility of Acme Textiles data.
- 4. Verify Role Restriction: Log in as Employee Ramesh Kumar (Company A). Notice the admin dashboard and sidebar links are gone; only "My Payslips" and "Ask AI" are accessible.
- 5. Test Ask AI: In the Ask AI page, click the suggestion chip "What is Ramesh's net salary for October?". The assistant executes internal tenant tools and returns ¹47,769.24.

5. Pure Payroll Calculation Engine & Worked Example

The payroll engine is encapsulated inside a pure, deterministic function in server/src/utils/payrollCalculator.js. It has zero database dependencies, making it 100% unit-testable and mathematically deterministic.

Mathematical Formulas:

```
1. Loss of Pay (LOP) Days:
   lopDays = absentDays + (0.5 * halfDays) + unmarkedDays

2. Daily Compensation Rate:
   perDaySalary = roundMoney(monthlySalary / workingDays)

3. Loss of Pay (LOP) Amount:
   lopAmount = roundMoney(perDaySalary * lopDays)

4. Earned Gross Monthly Salary:
   grossSalary = roundMoney(monthlySalary - lopAmount)

5. Statutory Tax & PF Deduction (default 10%):
   deduction = roundMoney(grossSalary * (deductionPercentage / 100))

6. Disbursable Net Salary:
   netSalary = roundMoney(grossSalary - deduction)
```

Worked Example (Ramesh Kumar - October 2026):

- Inputs: Monthly CTC = ¹60,000.00 | Working Days = 26 | Present = 22 | Absent = 2 | Half-days = 2
- 1. lopDays = 2 + (0.5 * 2) = 3.0 days
- 2. perDaySalary = 60,000 / 26 = ¹2,307.69 per day
- 3. lopAmount = 2,307.69 * 3.0 = ¹6,923.07
- 4. grossSalary = 60,000 - 6,923.07 = ¹53,076.93
- 5. deduction = 53,076.93 * 0.10 = ¹5,307.69
- 6. netSalary = 53,076.93 - 5,307.69 = ¹47,769.24

Unmarked Days Rule: Any working days not explicitly recorded as Present, Absent, or Half-day are treated as Absent, encouraging diligent HR attendance bookkeeping.

6. Multi-Tenancy Enforcement & RBAC Security

Security and tenant isolation form the core requirement of this system.

1. Why companyId Exclusively Originates from the JWT:

In insecure multi-tenant systems, client requests often pass tenant identifiers via URLs (/api/:companyId/payroll) or request bodies. Malicious users can tamper with these values to access rival organizations (Insecure Direct Object Reference - IDOR). In our system:

- When a user logs in, their companyId is cryptographically signed inside a tamper-proof JWT token.
- The authenticate middleware verifies the signature and injects req.user.companyId.
- Any companyId sent in request parameters, query strings, or JSON bodies is strictly ignored.

2. 404 Not Found vs 403 Forbidden Semantics:

If Company A admin attempts to fetch an employee or payroll record with an ID belonging to Company B:

```
// In server/src/controllers/employeeController.js
const employee = await User.findOne({
  _id: req.params.id,
  companyId: req.user.companyId, // Strictly scoped to caller's company!
});

if (!employee) {
  // Returns 404 (NOT 403) to prevent existence leakage of other tenants!
  return res.status(404).json({ success: false, message: 'Employee not found.' });
}
```

Returning 403 Forbidden would confirm to the attacker that the entity exists. Returning 404 Not Found completely obscures external records.

3. Role-Based Access Control (RBAC):

- Admin: Full authority to create employees, view all staff, and run payroll for their company.
- Employee: Restricted exclusively to their own profile and payslips. Requesting another colleague's payroll within the same company returns 403 Forbidden.

7. File-by-File Code Tour & Request Flow

A typical transaction flows cleanly through well-defined layers without leaky abstractions:

```
[React Component] (e.g. RunPayroll.jsx)
       %
       %¼ axios POST /api/payroll/calculate (attaches Bearer JWT)
[server.js / app.js] (Helmet, CORS, Express JSON parser)
       %
       %¼
[middleware/auth.js] (Verifies JWT, extracts req.user & req.user.companyId)
       %
       %¼
[middleware/validator.js] (Validates presence, numeric ranges, ISO YYYY-MM)
       %
       %¼
[controllers/payrollController.js]
  • Queries Company for workingDaysConfig
  • Queries User with { _id, companyId: req.user.companyId }
  • Calls pure utils/payrollCalculator.js
  • Upserts Attendance with compound index { companyId, employeeId, month }
       %
       %¼
[Mongoose Models] (Validates schema types & compound uniqueness)
       %
       %¼
[MongoDB Atlas] (Persisted document snapshot)
```

Key Source Files & Responsibilities:

- server/src/utils/payrollCalculator.js: Pure mathematical calculation logic.
- server/src/middleware/auth.js: JWT verification and authorizeRoles guard.
- server/src/services/payrollQueryService.js: Internal company-scoped data queries used by REST API and Ask AI.
- client/src/context/AuthContext.jsx: Client session lifecycle and auto-redirect on 401.
- client/src/pages/RunPayroll.jsx: Attendance steppers, validation bar, and printable payslip.

8. Bonus Natural Language Query Assistant

The POST /api/ask endpoint allows users to ask plain-English questions like "What is Ramesh's net salary for October?".

Zero Hallucination Architecture:

- The query service never invents or estimates compensation figures.
- Questions are resolved strictly by mapping to verified internal service functions (findEmployeeByName, getPayrollRecord).
- All query executions inherit the caller's verified req.user context: Company A cannot query Company B, and an employee asking about peers is refused with an access violation.
- Ambiguous names (e.g., multiple employees sharing a first name) prompt the user for clarification.

9. Assumptions, Known Limitations & Trade-offs

Assumptions Made:

- Unmarked Days: Any working day in a month not marked Present, Absent, or Half-Day is counted as Absent (Loss of Pay).
- Month Resolution: Plain English month names ("October") resolve to the current calendar year (2026).
- Deductions: Modeled as a flat 10% statutory rate (representing PF and standard tax withholdings).

Architectural Trade-offs:

- Shared Database, Shared Schema: Multi-tenancy is enforced via a indexed companyId column. This provides optimal operational efficiency and cost, but requires rigorous query filtering.
- Stateless Bearer JWT: Highly responsive for single-page applications. For banking-grade security, pairing with refresh token rotation in httpOnly cookies would be ideal.

10. Likely Interview Questions & Model Answers

Q1: Why did you choose a shared-database tenant architecture instead of a separate database per company?

Answer: "For a multi-tenant SaaS with hundreds of small-to-medium businesses, shared-database with a tenant column and compound indexes provides optimal connection pooling, faster schema migrations, and manageable infrastructure costs. We enforce hard logical boundaries through verified JWT claims, Mongoose compound indexes, and 404 error masking. For high-tier enterprise clients with strict compliance needs, we could transition to database-per-tenant without rewriting business logic."

Q2: How do you prevent Company A from seeing Company B's data via the LLM /ask endpoint?

Answer: "The LLM is never given direct database credentials or a companyId parameter to pick from. Instead, the model invokes grounded tool functions that execute under the authenticated user's req.user session. The data access service automatically injects req.user.companyId into every MongoDB query, ensuring tenant isolation is maintained identically across both REST endpoints and natural language queries."

Q3: Why does requesting another company's employee ID return 404 instead of 403?

Answer: "To prevent enumeration attacks. If an attacker guesses an ID and receives 403 Forbidden, they know that the resource exists within the system. Returning 404 Not Found maintains complete opacity, giving attackers zero information regarding other tenants' internal IDs."

Q4: What would you improve in this project with more time?

Answer: "I would add: 1) Daily biometric punch-clocking with geolocation, 2) Comprehensive Indian tax slab calculations (Section 115BAC), 3) Exporting payroll ledgers to CSV and automated bank transfer files (NEFT/RTGS), and 4) Refresh token rotation with Redis revocation."

11. Production Cloud Deployment Blueprint

The application is production-ready for deployment to standard cloud platforms:

- Backend (Render / Railway): Deploy the /server directory. Set environment variables (MONGODB_URI, JWT_SECRET, CLIENT_URL, PORT). Start command: node src/server.js.
- Frontend (Vercel / Netlify): Deploy the /client directory. Set VITE_API_URL to the production backend URL. Build command: npm run build, Output directory: dist.
- Database: Hosted on MongoDB Atlas with IP whitelisting restricted to production server IPs.